Sponsored by Vercel. Supported by Fastly, Upstash, Neon, Mux, and 1Password. Hide this message

07
Design A Key-value Store

Understand the problem and establish design scope

Many of us prefer using popular open-source solutions, such as Redis and Memcached, to store key-value pairs because it takes too much time to implement a key-value store from scratch. It is a good idea to understand how to design a key-value store even if we choose to use third-party implementations.

In this chapter, we will design a key-value store that supports:

- The size of a key-value pair is small: less than 10 KB.
- High scalability. The system should be highly scalable, even large-scale.
- Automatic scaling: the addition of servers should be automatic.
- Tunable consistency. The system should be tunable to provide different levels of consistency.
- High availability: the system should handle even if it spans across many servers.
- Low latency.

Single server key-value store

Developing a key-value data store that resides in a single server is easy. It could be quite challenging to store all data in a single server, which will soon reach its capacity. A quick optimization that can be done for a single server key-value store is to compress data. It also takes some time to implement and test for a powerful server. The rest of this section assumes that one server is not enough to store key-value pairs.

Distributed key-value store

A distributed key-value store is also known as a distributed hash table, which distributes key-value pairs across many servers. While designing a distributed system is quite challenging, it is worthwhile to discuss some useful properties and techniques that are used in such a system. Some of the properties and techniques may be familiar to you because they are used in many different systems. We will discuss how they are used in a key-value store in this chapter.

CAP theorem

CAP theorem is a useful concept when we consider what properties we can have in a distributed system. It is a useful tool to help us understand what properties we can have in a distributed system. The following definitions are used in this chapter.

Consistency: consistency means all clients see the same data at the same time no matter which node they connect to.

Availability: availability means any client which requests data gets a response even when some nodes are down.

Partition Tolerance: a partition is a communication break between two nodes. Partition tolerance means the system continues to operate despite network partitions.

CAP theorem states that it is impossible to simultaneously provide more than two of these three guarantees: consistency, availability, and partition tolerance. At least one must be sacrificed to gain the other two.

Consistency, availability, and partition tolerance are the three most important properties for a distributed key-value store. Some of the properties and techniques may be familiar to you because they are used in many different systems. We will discuss how they are used in a key-value store in this chapter.

Ideal system

In the ideal world, where partition does occur, there should not be any interruption of service. We could guarantee such a system. Unfortunately, it is not possible in the real world.

Figure 1

Real world distributed system

In a distributed system, partition tolerance is unavoidable since a network outage can last for an unpredictable amount of time. We must address partition tolerance. We can choose between consistency and availability in the case of a network partition. Some companies like to make a business decision to sacrifice one for the other.

Figure 2

If we choose consistency over availability (CP system), we have to stop some servers to accept writes in order to avoid data inconsistency among replicas. Banking software is a typical example of a CP system. Suppose we have a network partition in a banking system, we may need to stop some of the partitions to accept transactions because we want to avoid data inconsistency among replicas. Otherwise, different versions of a customer's data may be created in different servers, which is not acceptable for a banking system.

If we choose availability over consistency (AP system), the system must accept writes and data will be stored across servers even if it is not yet certain that the data will be saved for the long term or if it will be consistent in the future. Social media software is a typical example of an AP system. If a network partition happens, we can still let people post their new pictures. It is okay if we do not have the most recent copy of someone's profile picture for a while.

System components

In this section, we will discuss the following core components and techniques used to build a key-value store:

- Data partition
- Data replication
- Consistency
- Inconsistency resolution
- Handling failures
- Tombstones
- Tunable consistency
- Write path
- Read path

The ideas in Dynamo and Cassandra papers have been very popular in the industry. We will discuss many of these ideas in this chapter.

Data partition

For large applications, it is impossible to fit the complete data set in a single server. One simple way to overcome this bottleneck is to split the data into smaller partitions and distribute it amongst many servers. We can then be confident that we can store the data in the cluster.

The following are some of the benefits we can achieve with data partition:

- The size of each server is limited to the size of its hardware.
- Each server can handle a portion of the data.
- Adding more servers allows the system to handle more data.

Consider the following program where it is given an id.

Figure 3

With a hash table, we find the location of the key by hashing the key. For example, we can use the modulo operation to find the server where the data should be stored. For example, if we have four servers in the cluster, the modulo operation will return a value between 0 and 3 (inclusive). The client can then contact the server to read or write the data.

Consistent hashing

Since data is spread on multiple nodes, it is not the guaranteed server replica. Second, consider an scenario where the number of servers changes because a server goes offline.

The following are two problems with the hash table approach:

- If a server goes offline, the hash table will be updated to remove the unavailable server and redistribute the keys to other servers.
- Adding a new server will cause the hash table to be updated and redistribute keys.

Consistent hashing is a useful distribution scheme that does not rely on the number of servers. Adding and removing servers only affects a small number of keys. This allows us to add servers without affecting most of the keys. This allows us to add servers without affecting most of the keys. This allows us to add servers without affecting most of the